REAL ESTATE AGENCY PORTAL REPORT

CIS 344

By: Ahmad Huzaifa

GITHUB REPOSITORY:

https://github.com/ahmadhuzaifa2428-source/FINAL-PROJECT-344.git

1. Project Overview

The purpose of this project was to design and develop a fully functional Real Estate Agency Portal using PHP, MySQL, HTML, and CSS. The application allows users to register, log in, browse properties, add listings, submit inquiries, and save favorite properties. The system was designed to simulate a real-world real estate platform where agents can manage listings while buyers and renters can explore available homes and contact agents directly through the website.

The main goal of the project was to demonstrate full-stack development skills, including database design, backend processing, frontend development, and integration between PHP and MySQL. Additionally, the project focuses heavily on implementing relational database concepts such as primary keys, foreign keys, SQL joins, stored procedures, triggers, and views for efficient data retrieval and management.

The project also demonstrates how role-based systems work in a real application. Agents have different permissions from buyers and renters. Agents are allowed to add and manage properties, while buyers and renters are able to browse listings, submit inquiries, and save favorite properties.

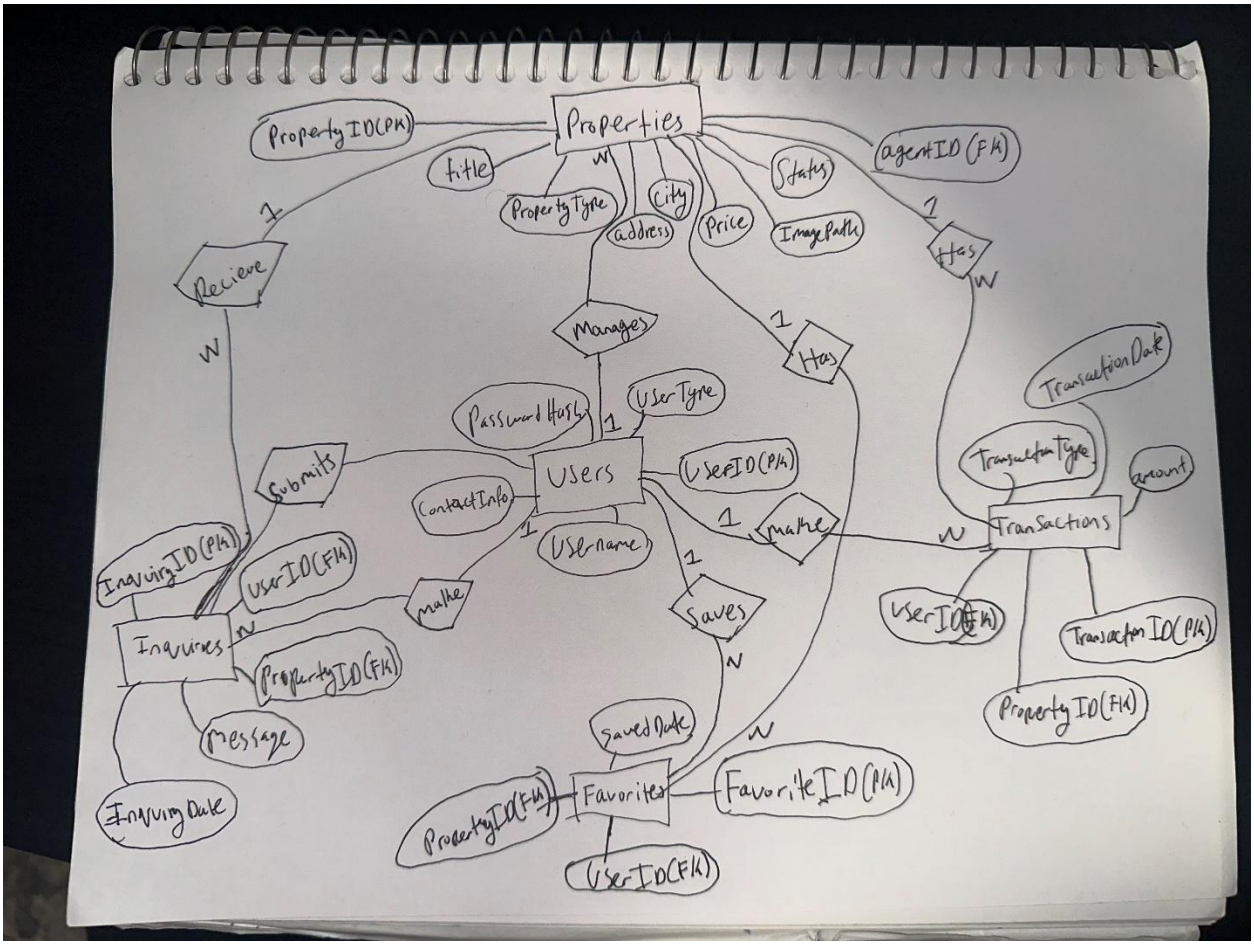
Another important goal of the project was improving the original starter files and transforming them into a more realistic and visually appealing website. The final website includes property cards, images, navigation menus, role-based dashboards, favorite properties, and improved CSS styling.

2. Development Process (Step-by-Step)

Step 1: EER Diagram Design

The project began with designing an Entity Relationship (EER) diagram. This diagram defined the structure of the database, including entities such as Users, Properties, Inquiries, Transactions, and Favorites. Relationships were established to show how agents manage properties, how buyers and renters submit inquiries, and how favorite properties are connected to users.

This step ensured that the database structure was logical before any coding began. It also helped us understand how all the tables would connect together later in the project through foreign key relationships.



One of the main struggles during this step was understanding how foreign keys actually connect tables together in a relational database. At first, some relationships were confusing because multiple tables depended on each other. For example, the Properties table connects to the Users table through the agentId field, while the Inquiries and Favorites tables connect both users and properties together.

I overcame this challenge by carefully reviewing our EER diagram and matching every foreign key to the correct primary key before building the database in phpMyAdmin.

Step 2: Writing SQL Code Manually

After creating the tables visually in phpMyAdmin, I moved on to writing the SQL code manually. Even though phpMyAdmin can automatically create tables visually, writing the SQL manually was important because it helped me understand exactly how the database works behind the scenes.

I wrote full CREATE TABLE statements for each table, including:

- Data types
- Primary keys
- Foreign keys
- Relationships
- Constraints

I also implemented SQL joins directly into the project. One important example appears in the properties page, where we combine property information together with the agent username. Without SQL joins, the website would only display numeric IDs instead of meaningful information.

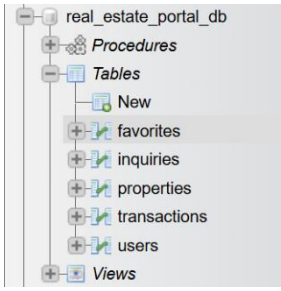
Another important database feature added into the project was the Favorites system. Buyers and renters are able to save favorite properties, and those favorites are stored inside the Favorites table using foreign key relationships.

The project also includes:

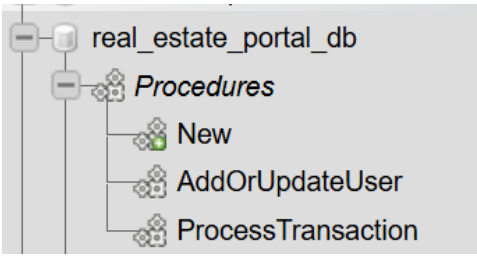
- Stored procedures
- A trigger
- A view

The trigger automatically updates property statuses after transactions are inserted. If a property is sold or rented, the status changes automatically inside the database.

phpMyAdmin Database:



Stored Procedures:



TRIGGER:

Trigger	Event	Table	Statement	Timing	Created
AfterTransactionInsert	INSERT	transactions	BEGIN IF NEW.transactionType = 'sale' THEN ...	AFTER	2026-05-07 17:40:22.

View:

One of the biggest struggles during this step was understanding SQL joins in practice and connecting them properly to the PHP files. At first, some pages only displayed IDs instead of useful information, while other pages returned no data at all.

I overcame this by testing each SQL query directly inside phpMyAdmin before placing it into the PHP files. This helped me confirm that the joins worked correctly before connecting them to the frontend.

Another issue happened when the website was tested on a new device. Even though the SQL logic was correct, no property listings appeared because the database had no starting data.

I solved this by adding preset sample accounts and property listings directly into the SQL file so the project always has usable data after importing the database.

3. Application Files and Functionality

1. index.php

This file acts as the homepage of the website. It introduces the Real Estate Agency Portal and guides users to browse properties, register, or log in.

The homepage was heavily redesigned from the original starter project. Property images, cards, buttons, navigation menus, and improved layouts were added to make the site feel more realistic.

Struggles:

At first, the homepage looked extremely plain and did not look like a real real estate website.

Overcome:

I improved the design using CSS, images, spacing, hover effects, and property cards to create a cleaner and more professional appearance.

2. register.php

This file allows users to create accounts.

Users can register as:

- Agent
- Buyer
- Renter

The registration system stores user information securely inside the Users table.

Struggles:

I initially had issues with duplicate usernames and invalid form submissions.

Overcome:

I fixed this by adding validation and UNIQUE constraints inside the database.

3. login.php

This file handles user login and authentication.

The system uses:

- password_hash()
- password_verify()
- PHP sessions

Preset Accounts:

Agent Account
Username: agent_maria
Password: password123

Buyer Account
Username: buyer_james
Password: password123

Renter Account
Username: renter_lisa
Password: password123

Struggles:

The preset accounts originally could not log in because the password hashes inside the database did not match the passwords.

Overcome:

I updated the password hashes properly so all accounts could log in successfully using password123.

4. Application Files and Purpose

index.php

Purpose: This file acts as the homepage of the website and introduces users to the Real Estate Agency Portal.

Struggle: The homepage originally looked very plain and did not feel like a real real estate website.

How I Overcame It: I redesigned the homepage using property cards, images, buttons, spacing, and improved CSS styling.

login.php

Purpose: This file handles user login and authentication.

Struggle: The preset accounts originally could not log in because the password hashes did not match.

How I Overcame It: I updated the password hashes inside the database and tested the login system multiple times.

register.php

Purpose: This file allows users to create new accounts.

Struggle: Duplicate usernames caused registration problems.

How I Overcame It: I added validation and UNIQUE constraints inside the Users table.

logout.php

Purpose: This file logs users out of the website.

Struggle: Users originally stayed logged in after clicking logout.

How I Overcame It: I used session_destroy() correctly to end user sessions.

dashboard.php

Purpose: This file displays the user dashboard after login.

Struggle: Some dashboard options originally appeared for the wrong user roles.

How I Overcame It: I added role-based session checks to control dashboard access.

properties.php

Purpose: This file displays all property listings from the database.

Struggle: Property images originally appeared blank.

How I Overcame It: I corrected the image paths and organized the images inside the assets/images folder.

property_details.php

Purpose: This file displays detailed information about one selected property.

Struggle: Some property information was not displaying properly because the SQL joins were incomplete.

How I Overcame It: I corrected the SQL joins and verified the property ID logic.

add_property.php

Purpose: This file allows agents to add new properties into the system.

Struggle: Buyers and renters originally could access the Add Property page.

How I Overcame It: I implemented role-based access restrictions so only agents can add properties.

submit_inquiry.php

Purpose: This file allows buyers and renters to send property inquiries.

Struggle: Some inquiries failed because the property ID was not passing correctly through the URL.

How I Overcame It: I corrected the GET and POST logic and tested the inquiry system repeatedly.

favorites.php

Purpose: This file displays saved favorite properties for the logged-in user.

Struggle: Favorite properties originally did not display properly for different users.

How I Overcame It: I connected the favorites query directly to the current logged-in user session.

save_favorite.php

Purpose: This file saves properties into the Favorites table.

Struggle: Saving the same property twice caused duplicate SQL errors.

How I Overcame It: I added validation to check if the property already existed before inserting it.

config/config.php

Purpose: This file stores database settings and starts PHP sessions.

Struggle: Database connection settings originally did not match the XAMPP environment.

How I Overcame It: I corrected the database credentials and verified the database name.

classes/Database.php

Purpose: This file creates the PDO database connection.

Struggle: PDO connection errors occurred during testing.

How I Overcame It: I corrected the DSN and database configuration.

classes/RealEstateDatabase.php

Purpose: This file contains the main database functions used throughout the project.

Struggle: The favorites system originally could not access the PDO connection.

How I Overcame It: I added a getConnection() method to return the database connection.

includes/header.php

Purpose: This file contains the website navigation bar and header section.

Struggle: The Favorites page originally did not appear in the navigation menu.

How I Overcame It: I added a Favorites link into the navigation bar.

includes/footer.php

Purpose: This file contains the footer section used across the website.

Struggle: The footer originally had to be repeated on every page.

How I Overcame It: I reused one shared footer file for cleaner code.

includes/auth.php

Purpose: This file protects pages that require login and role-based access.

Struggle: Different users needed different permissions.

How I Overcame It: I added role-checking functions and session validation.

assets/style.css

Purpose: This file controls the entire visual design of the website.

Struggle: The original starter website looked outdated and basic.

How I Overcame It: I redesigned the frontend using cards, spacing, colors, buttons, and property images.

real_estate_portal_db.sql

Purpose: This file creates the entire database structure and inserts sample data.

Struggle: The SQL file originally did not include all required project features.

How I Overcame It: I updated the SQL file to include preset accounts, sample properties, stored procedures, trigger, view, and favorites support.

4. Security Enhancements

Security was an important part of this project because users constantly enter data into the system, including usernames, passwords, property information, and inquiry messages.

One of the biggest security risks I focused on was SQL injection. Without proper protection, malicious users could attempt to manipulate the database through forms and potentially access or damage information inside the system.

To prevent this, I used prepared statements throughout the project. Prepared statements separate user input from SQL commands, making the website much safer because the database treats user input as data instead of executable SQL code.

Another important security feature was password hashing.

Instead of storing passwords as plain text, the project uses:

- password_hash()
- password_verify()

This ensures passwords are stored securely inside the database. Even if someone were able to access the database, they would not immediately see actual user passwords.

The project also uses PHP sessions for authentication and role-based access. Role-based access control was implemented to make sure different users only have access to the correct features.

Agents are allowed to:

- Add properties
- Manage listings

Buyers and renters are allowed to:

- Browse properties
- Submit inquiries
- Save favorites

One of the more difficult parts of the project was implementing the security features correctly. At first, I did not fully understand how password hashing, prepared statements, and PHP sessions worked together. Some login attempts failed, session information was not saving properly, and certain pages could still be accessed without logging in.

To better understand these concepts, I spent time researching documentation and watching YouTube tutorials about PHP authentication systems, SQL injection prevention, sessions, and role-based access control. I also tested different versions of the code multiple times until everything worked correctly.

Another challenge was understanding how prepared statements actually protect the database. Initially, I was writing simpler SQL queries directly into the PHP files, but I later learned that prepared statements are much safer and are considered standard practice in real-world applications.

I also had to carefully test the role-based system to make sure buyers and renters could not access agent-only pages like add_property.php. This required additional session validation and permission checks across multiple files.

Overall, this part of the project helped me understand how important security is in web development. Even though this is a student-level application, implementing these features

gave me a much better understanding of how real websites protect user information and prevent unauthorized access.

5. User Guide

PART A: Running the Website

Step 1: Install XAMPP

Install XAMPP with:

- Apache
- MySQL

Step 2: Place Project Folder

Place the project folder inside:

C:/xampp/htdocs

The folder should appear as:

C:/xampp/htdocs/real_estate_portal_

Step 3: Start Apache and MySQL

Open XAMPP and start:

- Apache
- MySQL

Step 4: Open phpMyAdmin

Go to:

http://localhost/phpmyadmin

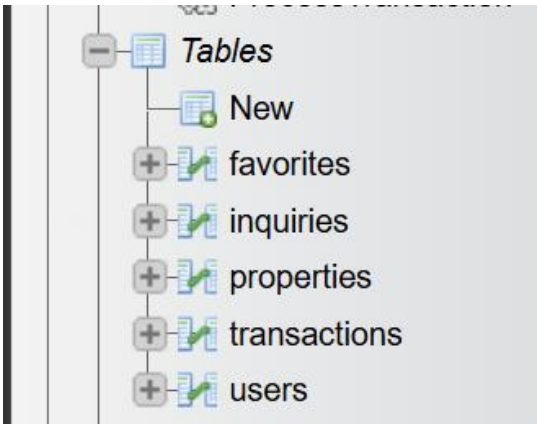
Step 5: Import SQL File

1. Click Import
2. Select real_estate_portal_db.sql
3. Click Go

Step 6: Verify Database Tables

Verify these tables exist:

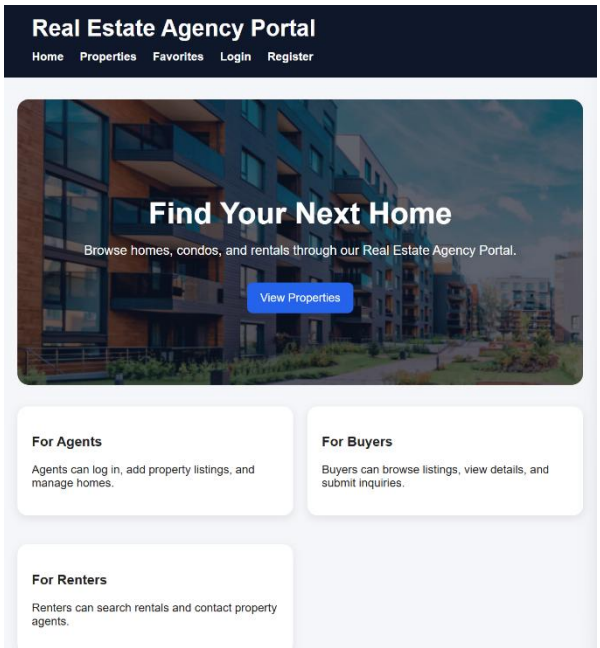
- Users
- Properties
- Inquiries
- Transactions
- Favorites



Step 7: Open the Website

Go to:

http://localhost/real_estate_portal_/



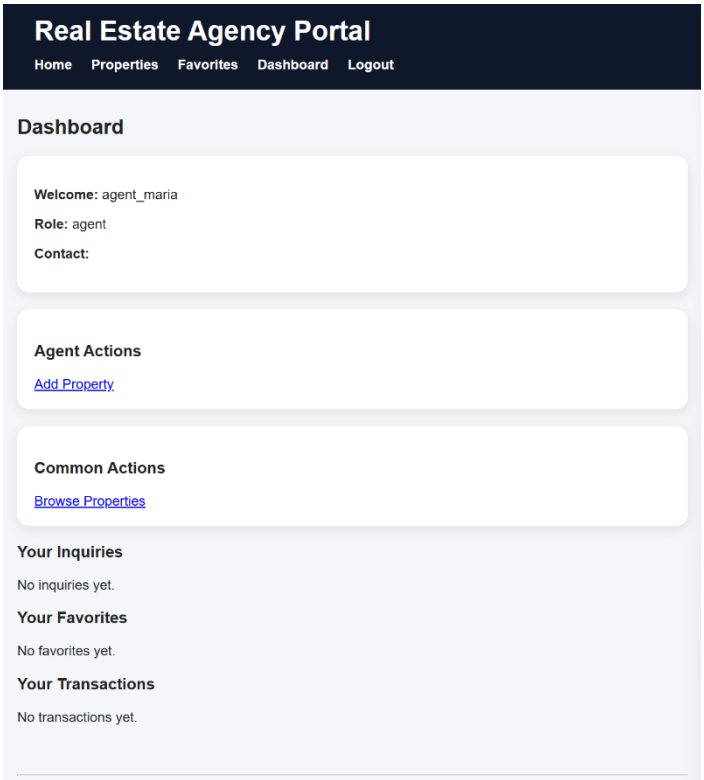
6. Agent Walkthrough

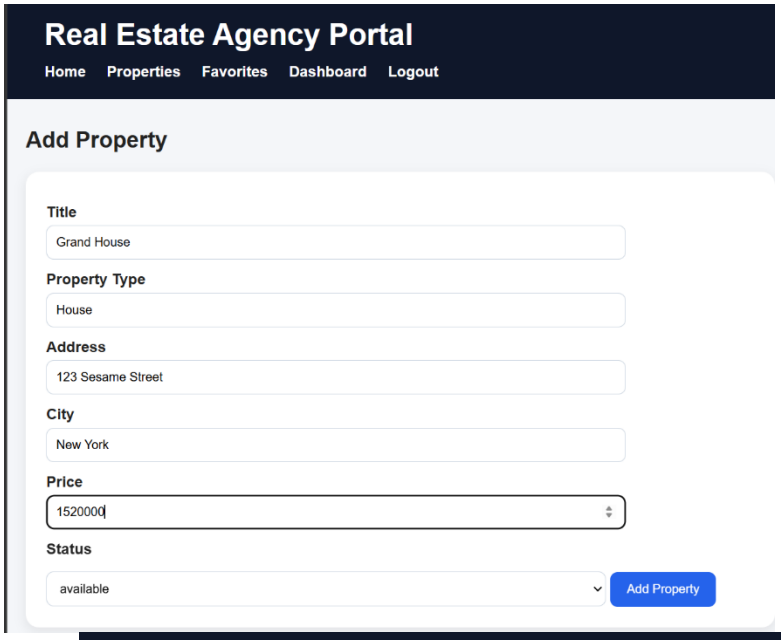
Login as Agent

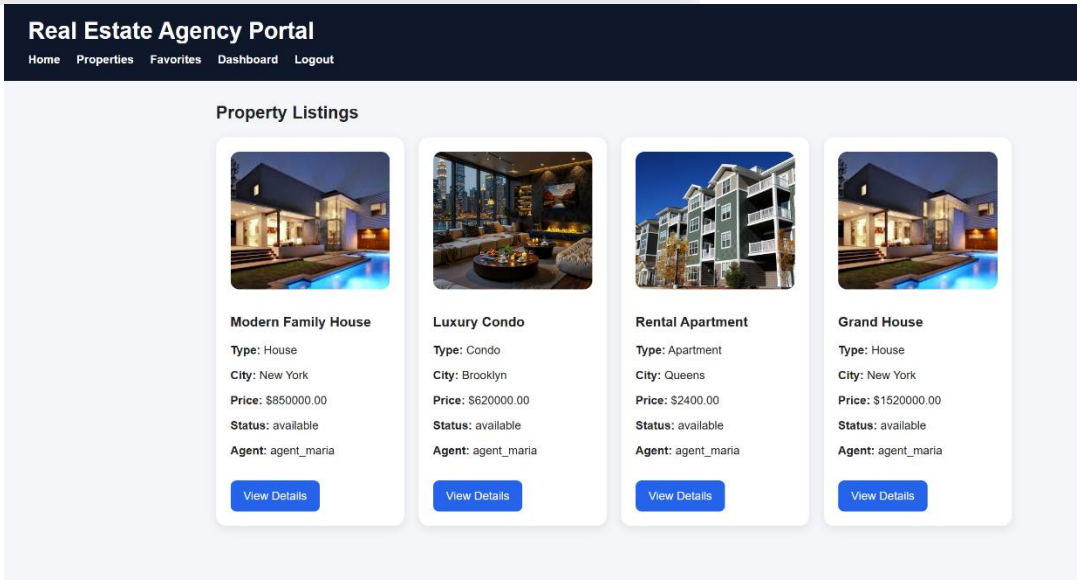
Username: agent_maria
Password: password123

After logging in, the agent can:

- Access dashboard
- Add properties
- View listings







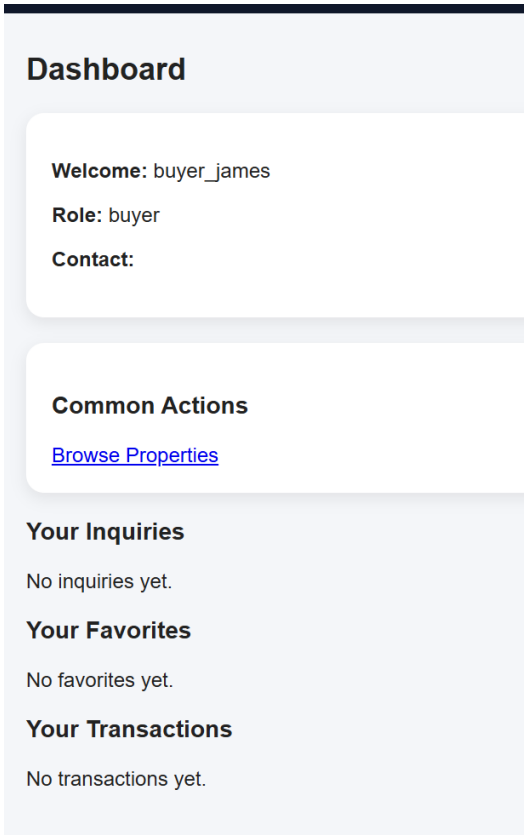
7. Buyer Walkthrough

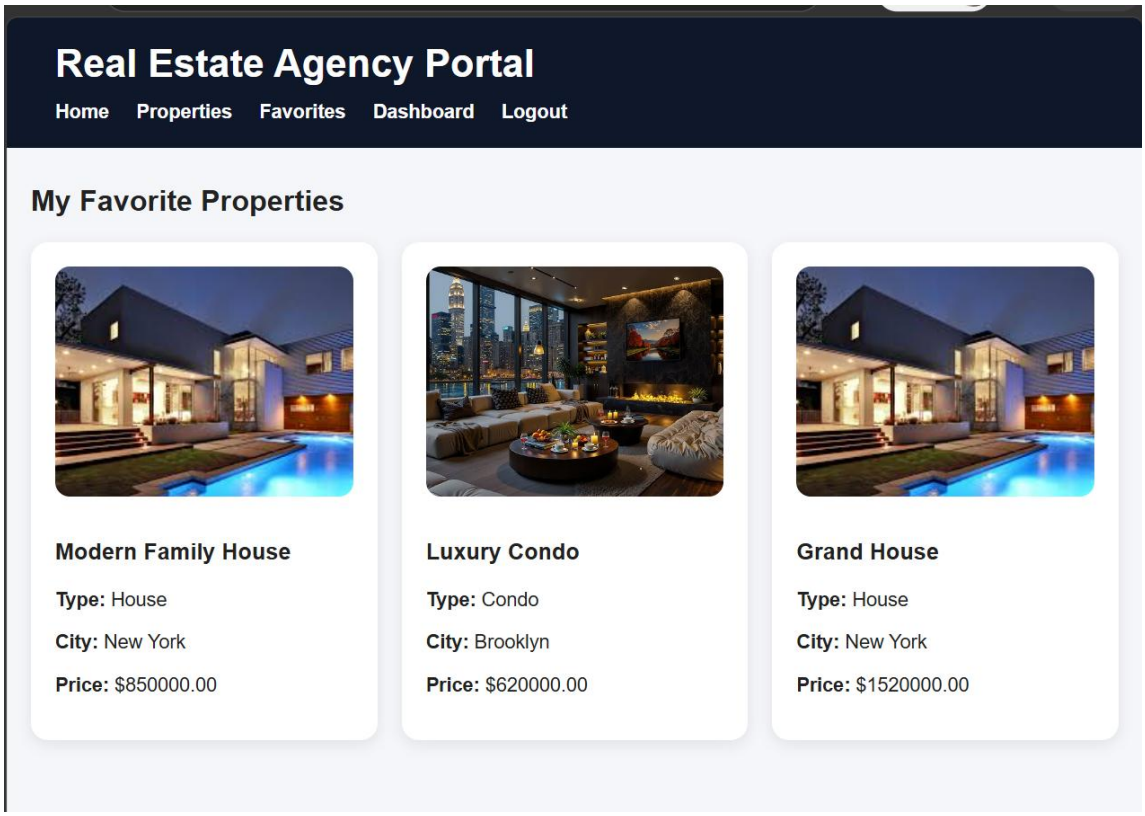
Login as Buyer

Username: buyer_james
Password: password123

The buyer can:

- Browse properties
- View details
- Submit inquiries
- Save favorite properties





8. Renter Walkthrough

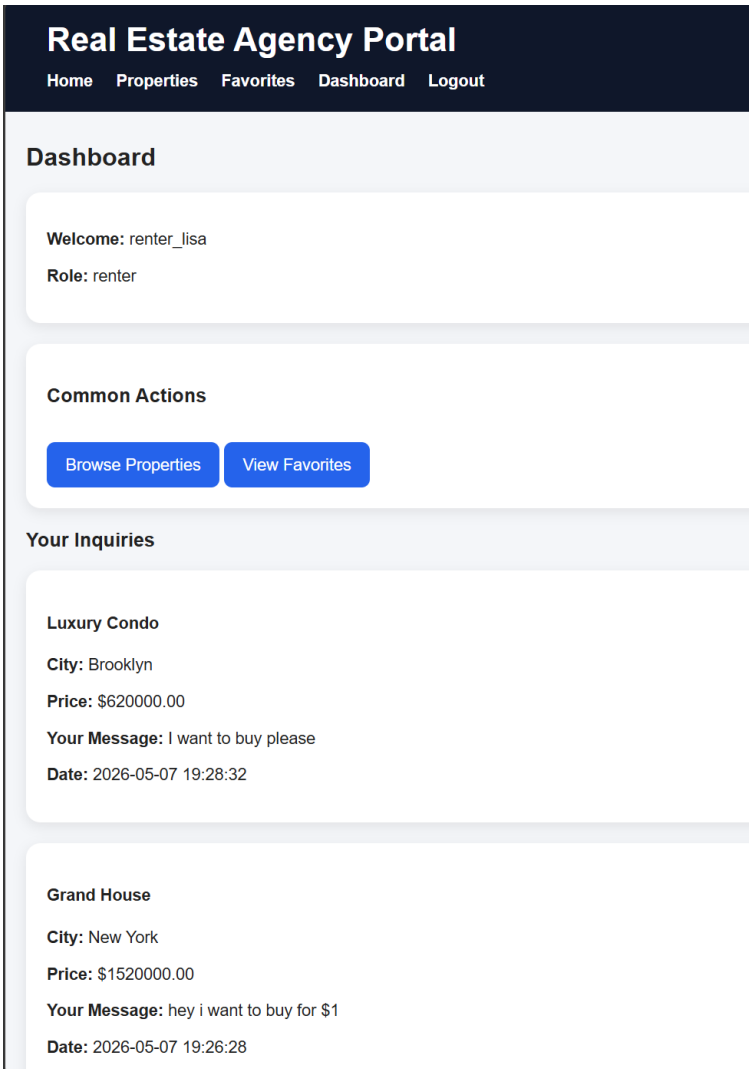
Login as Renter

Username: renter_lisa
Password: password123

The renter role works similarly to the buyer role but is intended for rental properties.

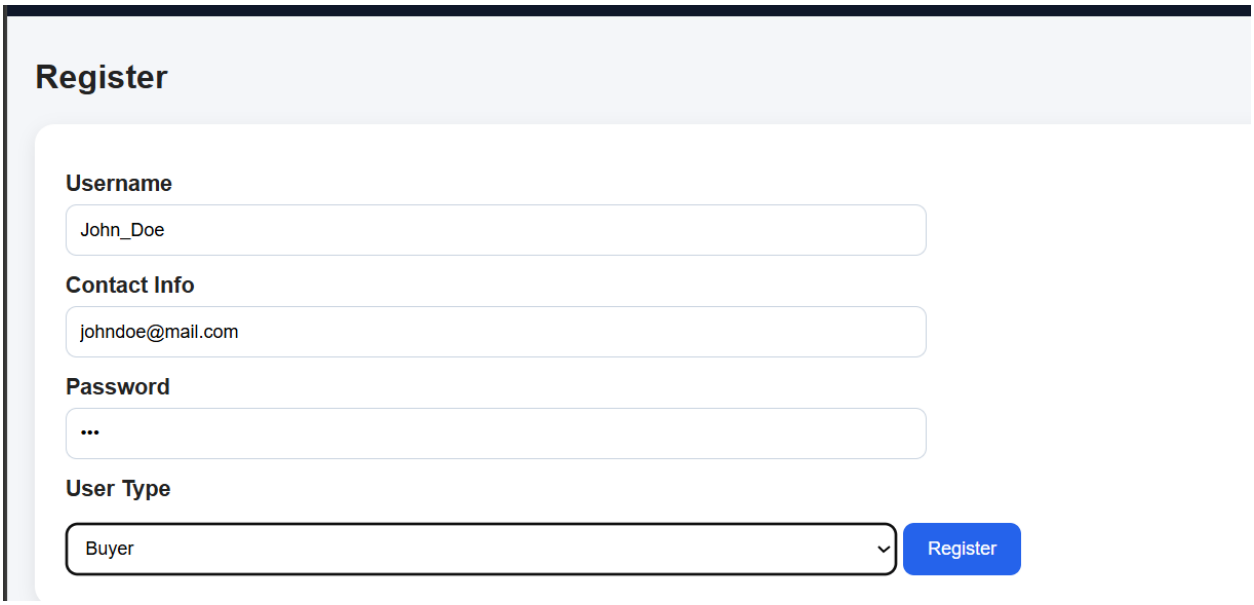
Renters can:

- Browse listings
- Submit inquiries
- Save favorite properties



9. Make Your Own Account

You can now follow up and make your own account whether it be, buyer, renter, or agent.



Step 10: Verify info via PHPMyAdmin

You can view all the information of what you did on the website by clicking on the databases and seeing if the actions got translated over.

	inquiryId	userId	propertyId	message	inquiryDate
Edit Copy Delete	1	2	1	I am interested in this property.	2026-05-07 18:34:06
Edit Copy Delete	2	3	2	Is this condo still available?	2026-05-07 18:34:06
Edit Copy Delete	3	2	3	Can I schedule a viewing?	2026-05-07 18:34:06
Edit Copy Delete	4	2	4	Hi Can i offer you $5?	2026-05-07 19:01:51
Edit Copy Delete	5	3	3	Hey i want to rent	2026-05-07 19:16:55
Edit Copy Delete	6	3	4	hey i want to buy for $1	2026-05-07 19:26:28
Edit Copy Delete	7	3	2	I want to buy please	2026-05-07 19:28:32

Extra options

	favoriteId	userId	propertyId	savedDate
Edit Copy Delete	1	2	1	2026-05-07 18:34:06
Edit Copy Delete	2	2	2	2026-05-07 18:34:06
Edit Copy Delete	3	3	3	2026-05-07 18:34:06
Edit Copy Delete	4	2	4	2026-05-07 19:03:17

	propertyId	title	propertyType	address	city	price	status	imagePath	agentId
Edit Copy Delete	1	Modern Family House	House	15 Main Street	New York	850000.00	available	assets/images/property1.jpg	1
Edit Copy Delete	2	Luxury Condo	Condo	88 Downtown Ave	Brooklyn	620000.00	available	assets/images/property2.jpg	1
Edit Copy Delete	3	Rental Apartment	Apartment	120 River Road	Queens	2400.00	available	assets/images/property3.jpg	1
Edit Copy Delete	4	Grand House	House	123 Sesame Street	New York	1520000.00	available	assets/images/property1.jpg	1

	userId	userName	contactInfo	passwordHash	userType
Edit Copy Delete	1	agent_maria	maria@email.com	$2y$12$YfA/rFsr3LMdkuPsJE5yeOSEmvRw/6.68da6luxebIy...	agent
Edit Copy Delete	2	buyer_james	james@email.com	$2y$12$YfA/rFsr3LMdkuPsJE5yeOSEmvRw/6.68da6luxebIy...	buyer
Edit Copy Delete	3	renter_lisa	lisa@email.com	$2y$12$YfA/rFsr3LMdkuPsJE5yeOSEmvRw/6.68da6luxebIy...	renter
Edit Copy Delete	4	John_Doe	johndoe@mail.com	$2y$10$K3ZhD3XNDz0xtClzkrZjSO8a0YztKE2MdecBL1TEDyT...	buyer

11. Conclusion

Overall, this project allowed me to combine everything learned about web development, databases, frontend design, and backend programming into one complete application.

The project started with an EER diagram and gradually developed into a fully working real estate portal connected to a MySQL database using PHP.

One of the most important parts of the project was understanding how relational databases function and how SQL joins connect information across multiple tables. Instead of storing everything in one table, we learned how to separate information properly and reconnect it when needed.

Another major takeaway from the project was understanding how frontend design and backend functionality work together. Small improvements such as property cards, images, hover effects, role-based dashboards, and favorite properties helped make the website feel significantly more realistic.

We also faced several development challenges including:

- Database connection problems
- Incorrect joins
- Broken image paths
- Login issues
- Duplicate favorite errors
- Foreign key relationship confusion

These challenges helped improve our debugging skills and taught us how to carefully test SQL queries, PHP logic, and frontend functionality.

Overall, the project successfully fulfills all assignment requirements, including relational database design, PHP backend development, frontend design, SQL joins, stored procedures, trigger implementation, role-based access control, and full MySQL integration.

Most importantly, this project provided a much stronger understanding of how full-stack applications are developed in real-world environments.